

CLASE PRÁCTICA

Login, Tokens y Firebase

Una guía práctica para implementar autenticación en aplicaciones **React** y **Expo** usando Firebase Auth y tokens JWT. Aprenderás el flujo completo: desde el registro hasta la protección de rutas.

REACT

EXPO

FIREBASE

JWT

¿Para qué sirve el login?

El login es la puerta de entrada de cualquier aplicación segura. Sin autenticación, cualquier usuario podría acceder a cualquier recurso sin restricciones.



Identificar al usuario

Saber quién está usando la aplicación en cada momento.



Proteger recursos

Evitar que usuarios no autorizados accedan a datos sensibles.



Controlar acceso

Definir qué puede ver y hacer cada tipo de usuario.



Personalizar la app

Ofrecer una experiencia única basada en las preferencias del usuario.

¿Qué incluye un sistema de autenticación?

Un sistema de autenticación completo no se limita al login. Estos son los cinco pilares que debes implementar en cualquier aplicación profesional:

01

Registro (Sign Up)

Crear una nueva cuenta de usuario con sus credenciales.

02

Login

Verificar identidad y obtener acceso a la aplicación.

03

Logout

Cerrar sesión de forma segura y limpiar el estado.

04

Persistencia de sesión

Mantener al usuario autenticado entre recargas o cierres de la app.

05

Validación en backend

El servidor verifica cada petición para garantizar la seguridad.

Funcionalidades comunes de auth

Gestión de cuenta

- Crear cuenta nueva
- Login con email/password
- Login con Google u otros proveedores
- Eliminar cuenta

Seguridad y verificación

- Logout seguro
- Recuperar contraseña olvidada
- Verificación por email
- Verificación OTP (código de un solo uso)



Firebase Auth soporta todas estas funcionalidades de forma nativa.

Flujo básico de autenticación

Este es el ciclo de vida de una sesión autenticada, desde que el usuario escribe sus credenciales hasta que el servidor responde con datos protegidos.



El **token** es el elemento central de este flujo. Una vez generado, actúa como la llave que abre cada endpoint protegido del backend.

TOKENS

¿Qué es un token?

Un **token** es una cadena de texto que identifica de forma única a un usuario autenticado. El backend lo genera al hacer login y el frontend lo almacena para enviarlo en cada petición posterior.

- Identifica al usuario sin necesidad de re-autenticar
- Se incluye en el header de cada request HTTP
- Tiene un tiempo de expiración configurable
- No contiene la contraseña del usuario

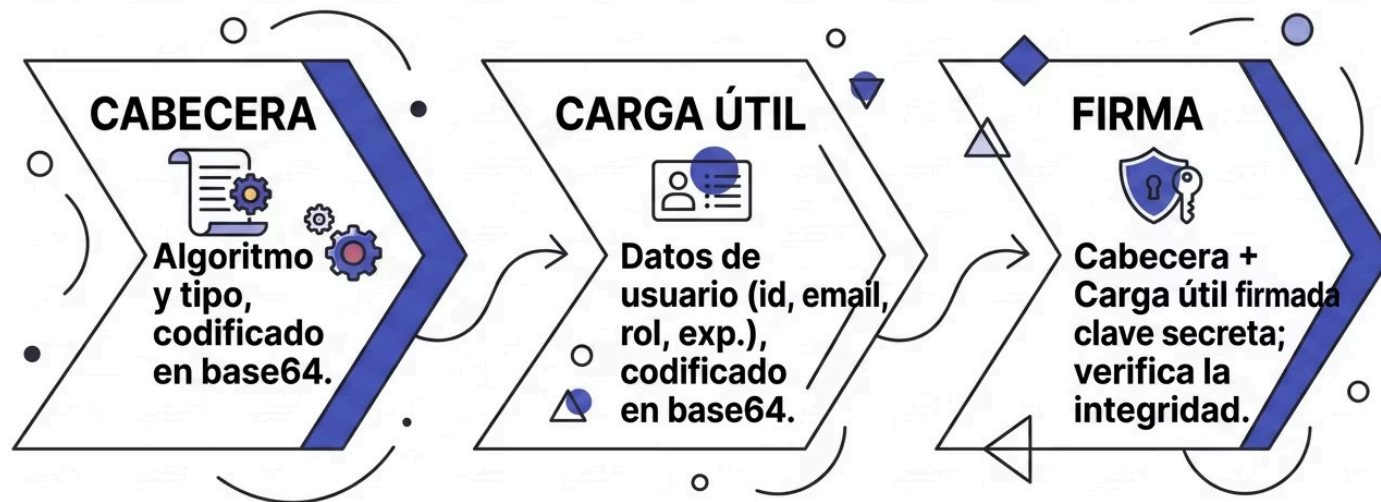
Formato en el header

Authorization: Bearer TOKEN

El prefijo **Bearer** es parte del estándar HTTP y debe incluirse siempre antes del token.

¿Qué es un JWT?

JSON Web Token (JWT) es el formato de token más utilizado en aplicaciones modernas. Está compuesto por tres partes separadas por puntos.



Firebase utiliza JWTs firmados con RS256. El backend puede verificar la firma sin necesidad de contactar a Firebase en cada petición.

¿Qué valida el backend?

Cuando el frontend envía un token, el backend realiza tres verificaciones críticas antes de responder con datos protegidos. Si cualquiera falla, la petición es rechazada.

✓ Token válido

La firma es correcta y el token no ha sido alterado. Se verifica usando la clave pública o secreta.

✓ Token no expirado

El campo `exp` del payload debe ser mayor al tiempo actual del servidor.

✓ Usuario autorizado

El usuario tiene permisos suficientes para acceder al recurso solicitado.

✗ Si cualquier verificación falla → el backend responde con **HTTP 401 Unauthorized**. El frontend debe manejar este error redirigiendo al login.

¿Dónde guardar el token?

La elección del almacenamiento impacta directamente en la seguridad de tu app. Cada opción tiene sus ventajas y limitaciones.

localStorage

- Persistente entre sesiones
- Fácil de usar
- ⚠ Vulnerable a XSS

sessionStorage

- Se borra al cerrar pestaña
- Igual de fácil que localStorage
- ⚠ También vulnerable a XSS

Cookies httpOnly

- No accesible desde JavaScript
- ✅ Más seguro contra XSS
- Más complejo de implementar

ⓘ ⚠ XSS (Cross-Site Scripting) es un tipo de ataque donde código JavaScript malicioso se inyecta en la página. Si el token está en localStorage o sessionStorage, ese script puede leerlo y robarlo.

¿Cuál usar en cada contexto?

Para esta clase

-  **localStorage** en aplicaciones web con React
-  **AsyncStorage** en proyectos con Expo / React Native

Son simples, funcionan perfectamente en entornos de desarrollo y facilitan el aprendizaje del flujo de tokens.

En producción real

-  **Cookies httpOnly** como estándar de seguridad
- No son accesibles desde JavaScript
- Protegen contra ataques XSS

 En apps críticas, nunca guardes tokens sensibles en localStorage en producción.

CÓDIGO

Guardar token en la web

En React para web, `localStorage` ofrece una API sencilla con tres operaciones esenciales para gestionar el token del usuario autenticado.

Guardar el token

```
localStorage.setItem("token", token);
```

Obtener el token

```
const token = localStorage.getItem("token");
```

Eliminar el token (logout)

```
localStorage.removeItem("token");
```

Cuándo usar cada operación

- **setItem** → justo después del login exitoso
- **getItem** → al enviar cualquier request autenticado
- **removeItem** → al hacer logout o al recibir un 401

Guardar token en Expo

En React Native / Expo, `localStorage` no existe. Se usa **AsyncStorage**, que funciona de forma asíncrona y persiste datos en el dispositivo móvil.

```
import AsyncStorage from "@react-native-async-storage/async-storage";

// Guardar token después del login
await AsyncStorage.setItem("token", token);

// Obtener token para enviarlo en requests
const token = await AsyncStorage.getItem("token");

// Eliminar token al hacer logout
await AsyncStorage.removeItem("token");
```

 Instala la dependencia con: `npx expo install @react-native-async-storage/async-storage`

Usar el token automáticamente con Axios

En lugar de agregar el token manualmente en cada llamada, los **interceptores de Axios** lo inyectan automáticamente en el header de cada request.

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem("token");  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`;  
  }  
  return config;  
});
```

- ✅ Con este interceptor, configuras el token **una sola vez** y todos los requests futuros lo incluyen automáticamente. No necesitas repetir código en cada llamada a la API.

Buenas prácticas con tokens

Seguir estas prácticas desde el inicio evita vulnerabilidades de seguridad comunes en aplicaciones en producción.

→ No hardcodear tokens ni secretos

Usa variables de entorno (.env) para guardar claves y configuraciones sensibles.

→ Limpiar el token en logout

Llama a `removeItem` al cerrar sesión para evitar accesos no autorizados.

→ Usar siempre HTTPS

Los tokens viajan en headers y pueden ser interceptados en conexiones HTTP sin cifrar.

→ Manejar la expiración del token

Detecta errores 401 y redirige al login cuando el token ya no es válido.

Firestore Auth: ¿qué es?

Firestore Authentication es un servicio de Google que resuelve la autenticación de usuarios sin necesidad de construir un backend propio. Gestiona el ciclo completo de forma segura y escalable.

- Servicio listo para usar sin configuración de servidor
- Maneja registro, login y gestión de sesiones
- Genera tokens JWT firmados automáticamente
- Soporta múltiples proveedores de identidad

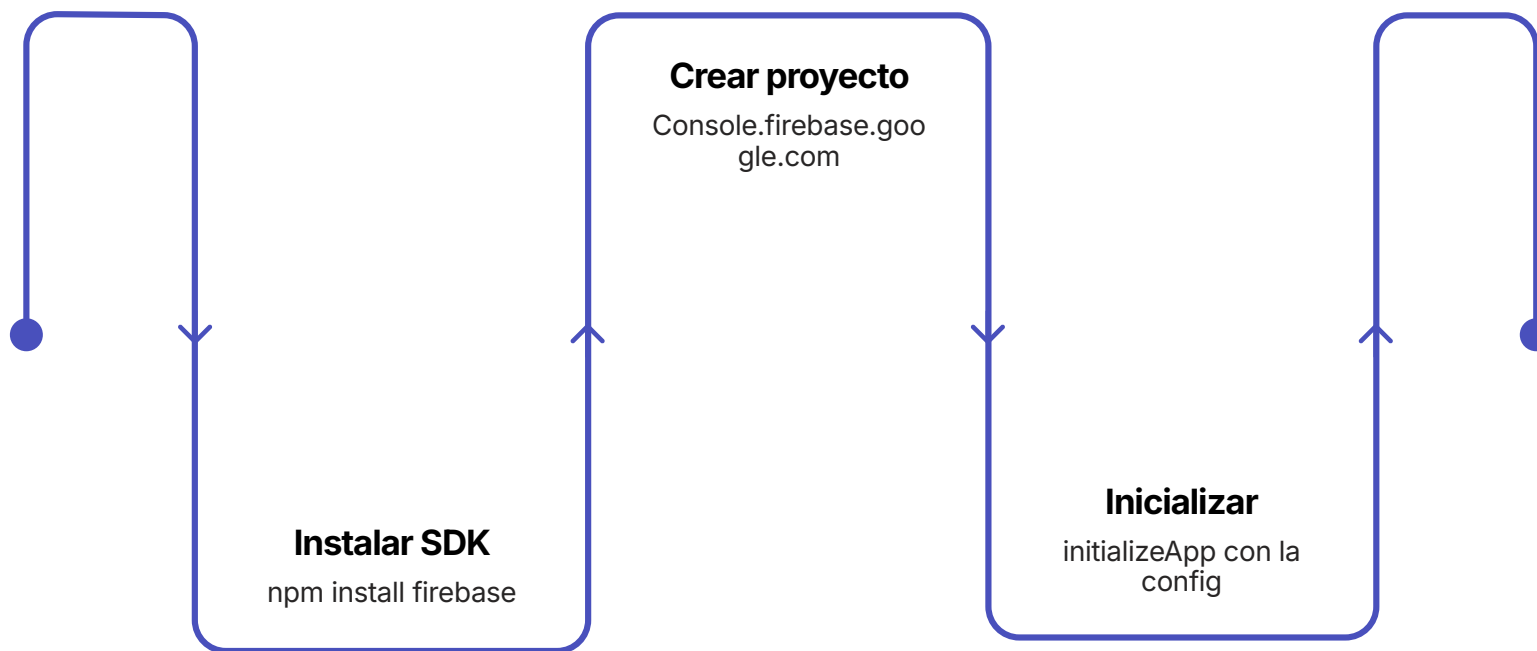
Proveedores soportados

-  Email y contraseña
-  Google
-  Facebook
-  GitHub
-  Teléfono (OTP)

Instalación de Firebase

Antes de escribir código, necesitas instalar el SDK de Firebase en tu proyecto. El proceso es el mismo para proyectos React web y Expo.

```
npm install firebase
```



 Crea tu proyecto en **console.firebase.google.com**, ve a Configuración del proyecto y copia el objeto `firebaseConfig` que se genera automáticamente.

Configuración básica de Firebase

Crea un archivo `firebase.ts` en tu proyecto para centralizar la configuración e inicialización de Firebase.

```
import { initializeApp } from "firebase/app";

const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "...",
};

export const app = initializeApp(firebaseConfig);
```

⚠ Nunca subas tu `firebaseConfig` real a un repositorio público. Usa variables de entorno con el prefijo `VITE_` o `EXPO_PUBLIC_`.

Inicializar Firebase Auth

Con la app de Firebase inicializada, el siguiente paso es obtener la instancia de autenticación. Esta instancia se reutiliza en todas las funciones de auth.

```
import { getAuth } from "firebase/auth";  
import { app } from "../firebase";  
  
export const auth = getAuth(app);
```

¿Por qué exportar auth?

Al exportar la instancia, puedes importarla directamente en cualquier función de login, logout o registro sin reinicializar Firebase cada vez.

Estructura recomendada

- `firebase.ts` → config + app
- `auth.ts` → instancia de auth
- `authService.ts` → funciones de login/logout

Login con email y contraseña

Esta función realiza el login completo con Firebase y devuelve el token JWT listo para ser almacenado y enviado al backend.

```
import { signInWithEmailAndPassword } from "firebase/auth";
import { auth } from "../firebase";

export const login = async (email: string, password: string) => {
  const userCredential = await signInWithEmailAndPassword(
    auth,
    email,
    password
  );
  const user = userCredential.user;
  const token = await user.getIdToken();
  return token;
};
```

✔ `getIdToken()` devuelve el JWT firmado por Firebase que el backend puede verificar sin ninguna llamada adicional a Firebase.

Guardar el token tras el login

Una vez que tienes el token de Firebase, el siguiente paso es guardarlo en `localStorage` (web) o `AsyncStorage` (Expo) para que el interceptor de Axios lo use en cada request.

```
// En tu componente o hook de login
const handleLogin = async () => {
  try {
    const token = await login(email, password);
    localStorage.setItem("token", token);
    navigate("/dashboard"); // redirigir al usuario
  } catch (error) {
    console.error("Error en login:", error);
    setError("Credenciales incorrectas");
  }
};
```

⚠ Siempre maneja los errores del login. Firebase lanza excepciones con códigos específicos como `auth/wrong-password` o `auth/user-not-found`.

Logout con Firebase

El logout debe hacer dos cosas: cerrar la sesión en Firebase **y** eliminar el token almacenado localmente. Si omites alguno de los dos pasos, la sesión queda en un estado inconsistente.

```
import { signOut } from "firebase/auth";
import { auth } from "../firebase";

export const logout = async () => {
  await signOut(auth);
  localStorage.removeItem("token");
};
```

signOut(auth)

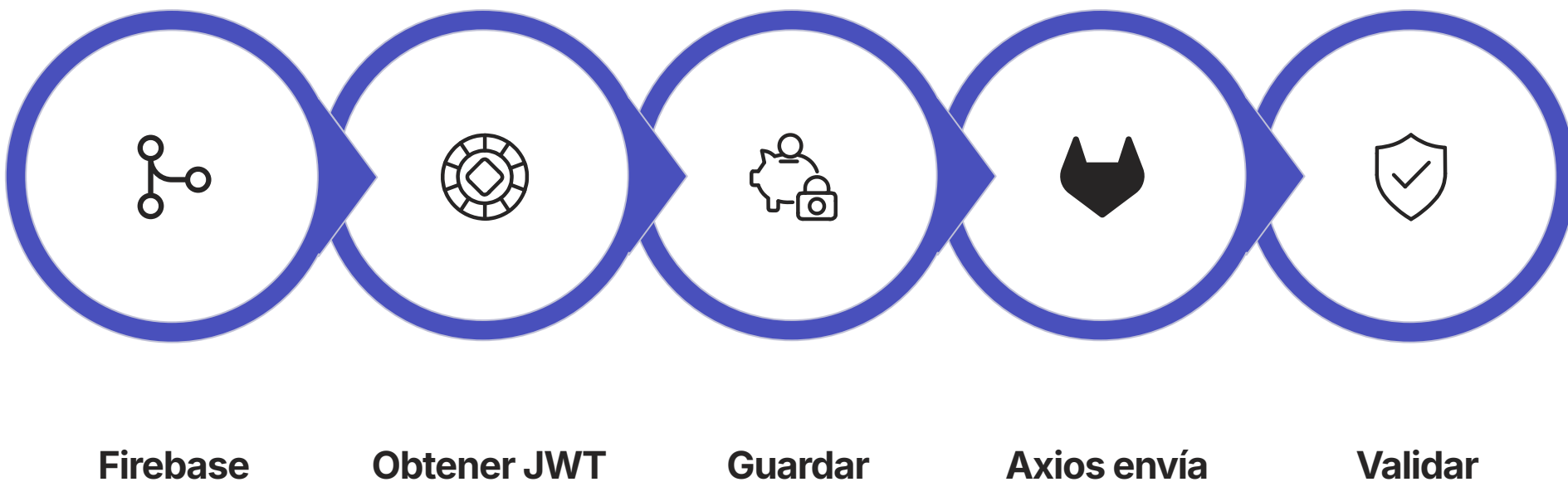
Invalida la sesión en Firebase y limpia el estado interno del SDK.

removeItem("token")

Elimina el token guardado localmente para que Axios deje de enviarlo.

Flujo completo en la app

Así se integran todos los componentes del sistema: Firebase, el token, Axios y el backend, formando un ciclo de autenticación completo y seguro.



Cada pieza tiene una responsabilidad clara: **Firebase** autentica, el **token** viaja, **Axios** lo gestiona y el **backend** decide si autoriza o rechaza.

Integración con el backend

Es fundamental entender qué hace cada parte del sistema para no confundir responsabilidades. La división de tareas es clara y debe respetarse.



Firestore genera el token

Firestore autentica al usuario y emite un JWT firmado con la clave privada de tu proyecto. Este token contiene el UID, email y más datos del usuario.



Frontend lo envía

El frontend almacena el token y lo adjunta automáticamente a cada request mediante el interceptor de Axios en el header `Authorization`.



Backend lo valida

El backend usa el SDK de Firestore Admin para verificar la firma del token y extraer la información del usuario sin necesidad de una base de datos propia de sesiones.

Proteger rutas en el frontend

Además de que el backend valide el token, el frontend debe evitar que usuarios no autenticados accedan visualmente a rutas protegidas.

Verificación simple

```
const token = localStorage.getItem("token");
if (!token) {
  navigate("/login");
}
```

Componente PrivateRoute

```
const PrivateRoute = ({ children }) => {
  const token = localStorage.getItem("token");
  if (!token) return <Navigate to="/login" />;
  return children;
};
```

Uso en el router

```
<Route path="/dashboard"
  element={
    <PrivateRoute>
      <Dashboard />
    </PrivateRoute>
  }
/>
```

Envuelve cualquier ruta privada con `PrivateRoute` para protegerla automáticamente.

Errores comunes al implementar auth

Estos son los errores más frecuentes entre desarrolladores que implementan autenticación con tokens por primera vez. ¡Evítalos desde el inicio!

1

No guardar el token

Hacer el login pero olvidar llamar a `setItem`. El usuario queda sin sesión.

2

No enviarlo en headers

Olvidar configurar el interceptor de Axios. El backend recibe requests sin autenticar.

3

Token expirado sin manejo

No detectar el error 401 ni redirigir al login cuando el token caduca.

4

No hacer logout completo

Cerrar sesión en Firebase pero dejar el token en `localStorage`.

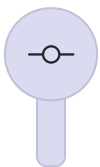
5

Usar mal Bearer

Escribir `bearer` en minúsculas o sin el espacio antes del token. El backend rechaza el header.

¿Qué van a implementar hoy?

En la actividad práctica de hoy aplicarás todos los conceptos vistos en clase para construir un sistema de autenticación funcional y conectado a un backend real.



Login con Firebase

Implementar la función de login usando `signInWithEmailAndPassword`.



Guardar el token

Almacenar el JWT en `localStorage` o `AsyncStorage` según la plataforma.



Integrarlo con Axios

Configurar el interceptor para enviar el token en cada request automáticamente.



Usar el token con Axios para conectar al backend

Usar el interceptor de Axios para enviar el token en cada request y obtener datos del usuario autenticado desde el backend.

OBJETIVO DEL DÍA

Objetivo de la actividad

Al finalizar la clase, deberás mostrar al profesor un sistema de autenticación completo y funcional. Esto contará como la actividad del día.

Firebase + Backend

Login con Firebase Auth integrado y comunicándose con tu API backend de forma segura.

ToDoList autenticada

Implementar login en el proyecto ToDoList que ya tienes. El usuario debe poder iniciar y cerrar sesión.

Demostración al profesor

Mostrar que el login funciona correctamente y que los requests al backend incluyen el token en el header.

👉 ¡Éxito! Si el login funciona, el token viaja en los headers y el backend responde correctamente, habrás completado el ciclo de autenticación completo. 🚀